DISEÑO DE VAGAX

NICOLAS JIMENEZ

JERÓNIMO LINARES

MIGUEL CELIS

ALEJANDRO LINDEMAN

LENGUAJES DE PROGRAMACIÓN

JOAQUIN FERNADEZ

UNIVERSIDAD SERGIO ARBOLEDA

1. INTRODUCCIÓN **3**
2. OBJETIVO **3**
2.1 Objetivo General 3
2.2 Objetivos Específicos 3
3. Justificación **4**
4. Descripción General del Sistema **4**
4.1 Dominio del Lenguaje 5
4.2 Paradigma del Lenguaje 5
4.3 Funciones de VAGAX 5
5. ARQUITECTURA DEL SISTEMA **5**
5.1 Capa de Entrada y Ejecución 5
5.2 Capa de Análisis Léxico y Sintáctico (ANTLR v4) 5
5.3 Capa de Interpretación (Visitor en Python) 6
6. Capa de Runtime Propio (Sin librerías matemáticas prehechas) **6**
6.1 Matemática (MATE) 6
6.2 Matrices (MATRX) 6
6.3 Visualización (VISU) 7
6.4 Archivos (IO) 7
6.5 Datos CSV (TABLAX) 7
6.6 Machine Learning (MLAB) 7
6.7 Deep Learning / Redes Neuronales (NEURAX) 7
7. DISEÑO DEL LENGUAJE (VAGAX) **8**
7.1 Principios de diseño 8
7.2 Elementos del lenguaje 8
7.2.1 Tipos de datos soportados 8
7.2.2 Estructuras de control (Control de flujo) 9
8. Gramática del Lenguaje VAGAX (Parser) **10**
8.1 Estructura general del programa 11
8.2 Definición de sentencias 11
8.3 Declaración de variables 12
8.4 Asignación de valores 13
8.5 Estructuras de control 13
8.5.1 Condicional IF 13
8.5.2 Ciclo WHILE 14
8.5.3 Ciclo FOR 14
8.6 Bloques de instrucciones 14
8.7 Definición de funciones 15
8.8 Retorno de valores 15
8.9 Expresiones 16
8.10 Llamadas a funciones 16
8.11 Literales 17
9. FUNCIONES **17**
10. Ejecución de llamadas a funciones **23**
10.1 Integración con el sistema de ejecución (main.py) 23

11. Conclusiones: 23
12. Referencias 24

1. INTRODUCCIÓN

El presente proyecto tiene como objetivo el diseño e implementación de un Lenguaje de Dominio Específico (DSL) orientado a procesos de Deep Learning, desarrollado con ANTLR v4 para el análisis léxico y sintáctico y ejecutado mediante el patrón Visitor en Python. El lenguaje permitirá realizar operaciones matemáticas, manipulación de matrices, estructuras de control, manejo de archivos y la implementación de algoritmos básicos de Machine Learning y Deep Learning.

De acuerdo con los lineamientos del proyecto, no se permite el uso de librerías matemáticas ni de aprendizaje automático ya existentes. Por esta razón, todas las funciones matemáticas, operaciones matriciales y modelos de aprendizaje serán implementados desde cero por el equipo.

La arquitectura del sistema estará dividida en tres componentes principales: el analizador generado con ANTLR, el intérprete basado en Visitor y un runtime propio que contendrá todas las implementaciones desarrolladas. Este enfoque garantiza una estructura organizada y coherente, cumpliendo estrictamente con los requisitos establecidos.

2. OBJETIVO

2.1 Objetivo General

Diseñar e implementar un Lenguaje de Dominio Específico (DSL) orientado a procesos de Deep Learning, utilizando ANTLR v4 para el análisis léxico y sintáctico, y el patrón Visitor para su ejecución, desarrollando desde cero todas las operaciones matemáticas, matriciales y algoritmos de aprendizaje automático, sin el uso de librerías externas.

2.2 Objetivos Específicos

1. Diseñar la gramática formal del DSL que permita definir variables, expresiones matemáticas, estructuras de control, funciones y operaciones matriciales.
2. Implementar el analizador léxico y sintáctico mediante ANTLR v4, generando el árbol de análisis correspondiente al lenguaje definido.
3. Desarrollar un intérprete basado en el patrón Visitor que recorra el árbol sintáctico y ejecuta cada instrucción del DSL.
4. Implementar desde cero funciones matemáticas fundamentales como potencias, raíces y funciones trigonométricas mediante métodos numéricos.

5. Desarrollar un módulo propio para el manejo de matrices que incluya operaciones como suma, multiplicación, transposición e inversión.
6. Implementar algoritmos básicos de Machine Learning y Deep Learning, como regresión lineal, clasificación logística, clustering y una red neuronal simple (MLP), sin depender de librerías especializadas.
7. Diseñar un sistema modular que permita la extensión futura del lenguaje y facilite el mantenimiento del código.

3. Justificación

El desarrollo del lenguaje “VAGAX” surge de la necesidad de contar con una herramienta especializada en la resolución de problemas de Deep Learning, pese a la existencia de lenguajes como Python, Java, C o Haskell, estos obligan a contar con muchos detalles técnicos para desarrollar operaciones, desarrollo de matrices, etc. Esto hace que en su mayoría, el código resulte ser menos claro y más difícil tanto de comprender como de relacionar con el problema que realmente se busca resolver.

Es por esto, que “VAGAX” se refiere como un lenguaje que permite representar de forma directa tareas de DL, su diseño busca acciones como realizar operaciones aritméticas, trabajar con matrices, crear condicionales y ciclos únicos, manejar archivos, generar gráficas, entre otras… Esto con motivo de que se puedan expresar a una sintaxis más cercana al dominio y que el usuario comprenda de forma más sencilla.

Además, el lenguaje sigue un enfoque funcional, lo que resulta ser adecuado para la organización de operaciones, reutilización de funciones y el hecho de mantener una estructura más clara dentro del tratamiento de datos. Esto con motivo de que el lenguaje no sea solo más coherente con el tipo de problemas que se buscan solventar, sino también que su diseño sea más ordenado.

Por otra parte, el uso de ANTLR y del patrón Visitor en Python, permite que la propuesta tenga una base formal dentro del diseño e implementación. Con base a esto y todo lo hablado, “VAGAX“ se justifica por su utilidad práctica dentro del dominio de DL como por su valor dentro el estudio de gramáticas, compiladores, interpretaciones y diseño de lenguajes.

4. Descripción General del Sistema

VAGAX es un Lenguaje de Dominio Específico (DSL) diseñado para modelar y ejecutar operaciones orientadas al Aprendizaje Profundo. Su propósito principal es ofrecer una sintaxis especializada que permita expresar de forma más clara, directa y organizada tareas que, en lenguajes de propósito general, suelen requerir múltiples estructuras adicionales, bibliotecas externas y mayor complejidad sintáctica.

4.1 Dominio del Lenguaje

El dominio de “VAGAX” se centra en operaciones matemáticas, manejo de matrices, estructuras de control, lectura y escritura de archivos de texto, generación de gráficas y definición de funciones con aplicaciones dentro de Deep Learning. Dentro de este alcance, el lenguaje contempla la representación de operaciones avanzadas en conjunto a distintas formas de análisis y de aprendizaje para la implementación que corresponda a todos los requisitos del lenguaje

4.2 Paradigma del Lenguaje

VAGAX, sigue un enfoque de paradigma funcional, tal cual se explica en el documento con los lineamientos del mismo, sin embargo este incorpora las estructuras de control propias de un lenguaje estructurado, tales como condicionales y ciclos, con el fin de proporcionar una flexibilidad superior al modelaje de procesos computacionales y los flujos de ejecución dentro del dominio.

4.3 Funciones de VAGAX

VAGAX contempla operaciones aritméticas, trigonométricas y matriciales, así como también lectura de archivos, graficación de datos, regresiones, construcción de modelos básicos de clasificación, diferente creación de librerías, agrupamiento y predicción de RNA. Todo será realizado utilizando ANTLR v4 y el Patrón Visitor en Python, como también Flex y Bison para una implementación con todos los requisitos y un funcionamiento exactamente como se solicita.

5. ARQUITECTURA DEL SISTEMA

La arquitectura se plantea en capas, para que el lenguaje sea fácil de mantener y ampliar, cumpliendo con lo que exige el documento: ANTLRv4, enfoque funcional, ejecución en consola/interfaz de texto, e implementación en Python usando Visitor.

5.1 Capa de Entrada y Ejecución

Responsabilidad: recibir el programa escrito en el DSL (desde un archivo o entrada directa) y ejecutar el flujo de compilación/interpretación.

- Modo archivo: deepLang archivo.dsl
- Modo interactivo: REPL (tipo terminal)

Esto cumple con que el lenguaje se ejecute por consola o interfaz de texto.

5.2 Capa de Análisis Léxico y Sintáctico (ANTLR v4)

Responsabilidad: transformar el texto del programa en una estructura navegable:

- Lexer: convierte caracteres → tokens
- Parser: convierte tokens → árbol de análisis (Parse Tree)

ANTLR v4 se usa para diseñar e implementar el lenguaje

5.3 Capa de Interpretación (Visitor en Python)

Responsabilidad: ejecutar el programa recorriendo el árbol.

- Expresiones matemáticas
- Asignaciones
- Condicionales
- Ciclos (for y while)
- Llamadas a funciones
- Manejo de variables y funciones definidas por el usuario

Aquí reside la lógica semántica del DSL.
Esta capa cumple el requisito de utilizar el patrón Visitor soportado por ANTLR.

6. Capa de Runtime Propio (Sin librerías matemáticas prehechas)

Responsabilidad: contener todas las implementaciones “pesadas” desarrolladas por ustedes (sin math y sin librerías de terceros), que el Visitor invoca.

Este runtime cubre las capacidades que pide el documento:

6.1 Matemática (MATE)

Encargado de las operaciones aritméticas y funciones matemáticas implementadas manualmente.

Incluye:

- Suma, resta, multiplicación y división
- Potencias y raíces (x^y)
- Funciones trigonométricas (*sin, cos, tan*)
- Métodos numéricos propios (por ejemplo, Newton-Raphson o series de Taylor)

6.2 Matrices (MATRX)

Encargado del manejo de álgebra lineal implementado desde cero.

Incluye:

- Suma y resta matricial
- Multiplicación matricial
- Transpuesta
- Inversa (método de Gauss-Jordan)
- Validación de dimensiones

6.3 Visualización (VISU)

Módulo encargado de la representación gráfica sin uso de bibliotecas externas.

Se implementará bajo un enfoque híbrido:

- Modo básico: visualización en consola mediante gráficos ASCII.
- Modo extendido: exportación de imágenes simples en formato PPM generadas manualmente.

Esto permite cumplir con el requisito de visualización de datos sin depender de herramientas como Matplotlib.

6.4 Archivos (IO)

Encargado del manejo de archivos de texto.

Incluye:

- Lectura de archivos
- Escritura de archivos
- Conversión de datos a estructuras internas del DSL

6.5 Datos CSV (TABLAX)

Encargado del manejo de archivos CSV.

Incluye:

- Lectura y separación por delimitadores
- Conversión de datos CSV a matrices internas
- Validación de estructura

Este módulo puede apoyarse internamente en IO.

6.6 Machine Learning (MLAB)

Implementaciones propias de algoritmos de aprendizaje automático.

Incluye:

- Regresión lineal
- Regresión logística
- Algoritmos de agrupamiento (ej. K-means)
- Algoritmos básicos de clasificación y predicción

6.7 Deep Learning / Redes Neuronales (NEURAX)

Módulo especializado en redes neuronales implementadas desde cero.

Incluye:

- Perceptrón simple
- Perceptrón multicapa (MLP)
- Propagación hacia adelante (forward)
- Retropropagación del error (backpropagation)
- Actualización de pesos mediante gradiente descendente
- Funciones de activación propias (ReLU, step, aproximaciones numéricas)

Este módulo depende internamente de:

- MATRX (operaciones matriciales)
- MATE (funciones matemáticas auxiliares)

7. DISEÑO DEL LENGUAJE (VAGAX)

VAGAX es un Lenguaje de Dominio Específico (DSL) orientado a procesos de Machine Learning y Deep Learning, diseñado para ejecutarse en consola y construido sobre ANTLR v4 con el patrón Visitor en Python.

El lenguaje se integra con un runtime interno propio, compuesto por los módulos MATE, MATRX, VISU, IO, TABLX, MLAB y NEURAX, los cuales implementan desde cero todas las funcionalidades matemáticas, matriciales, de visualización y aprendizaje automático, sin depender de librerías externas.

7.1 Principios de diseño

- Enfoque funcional: se prioriza el uso de funciones y expresiones como base del lenguaje, permitiendo construir soluciones de forma clara y modular.
- Sintaxis propia con identidad VAGAX: el lenguaje incorpora palabras reservadas con el sufijo VAG (ej. IFVAG, ELSEVAG, PRINTVAG), otorgándole identidad propia sin perder claridad estructural.
- Sintaxis estructurada y clara: bloques delimitados por llaves {}, uso explícito de paréntesis en condiciones y llamadas a funciones.
- Modularidad mediante runtime interno: operaciones complejas se invocan desde el lenguaje, pero son ejecutadas por módulos internos especializados (MATE, MATRX, VISU, etc.).
- Sin dependencias externas: todas las operaciones matemáticas, de álgebra lineal, visualización y redes neuronales son desarrolladas por el equipo desde cero.

7.2 Elementos del lenguaje

7.2.1 Tipos de datos soportados

VAGAX maneja un sistema de tipos estáticos con validación en tiempo de ejecución.

- Number: representa valores numéricos enteros y decimales.
- Bool: valores booleanos representados como SISAS y NOKAS.
- String: cadenas de texto delimitadas por comillas dobles.
- Matrix: estructuras bidimensionales representadas como listas anidadas con formato [[...],[...]].

- Function: funciones definidas por el usuario mediante FNVAG o funciones internas del sistema.
- Null: valor especial utilizado cuando una operación no retorna resultado explícito.

7.2.2 Estructuras de control (Control de flujo)

VAGAX soporta control de flujo mediante condicionales y ciclos, usando palabras reservadas con sufijo VAG. Estas estructuras permiten ejecutar bloques de instrucciones delimitados por llaves { }.

a) Condicionales: IFVAG y ELSEVAG

Permiten ejecutar un bloque si una condición es verdadera y, opcionalmente, otro bloque si es falsa.

Sintaxis:

```
1   IFVAG (condicion) {
2       // bloque verdadero
3   } ELSEVAG {
4       // bloque falso
5   }
```

Ejemplo:

```
x = 10;

IFVAG (x > 5) {
  PRINTVAG("x es mayor que 5");
} ELSEVAG {
  PRINTVAG("x es menor o igual a 5");
}
```

b) Ciclo WHILEVAG

Ejecuta repetidamente un bloque mientras la condición se mantenga verdadera.

Sintaxis general:

```
10
11  WHILEVAG (condicion) {
12      // bloque
13  }
```

Ejemplo:

```
i = 0;

WHILEVAG (i < 3) {
  PRINTVAG(i);
  i = i + 1;
}
```

c) Ciclo FORVAG

Permite un ciclo controlado por tres partes: inicialización, condición y actualización.

Sintaxis general:

```
FORVAG (inicio; condicion; actualizacion) {
  // bloque
}
```

Ejemplo:

```
FORVAG (i = 0; i < 5; i = i + 1) {
  PRINTVAG(i);
}
```

8. Gramática del Lenguaje VAGAX (Parser)

La gramática del lenguaje VAGAX define formalmente la estructura sintáctica que deben seguir los programas escritos en el DSL. Esta gramática fue implementada utilizando ANTLR v4, mediante un archivo de parser que describe las reglas de producción del lenguaje. Su función principal es transformar la secuencia de tokens generada por el analizador léxico en un árbol sintáctico (Parse Tree) que posteriormente será recorrido por el intérprete implementado en Python mediante el patrón Visitor.

El parser trabaja en conjunto con el lexer previamente definido. Mientras el lexer se encarga de identificar los tokens del lenguaje (palabras reservadas, operadores, identificadores y literales), el parser establece cómo estos tokens pueden combinarse para formar estructuras válidas dentro del lenguaje.

La gramática del parser fue diseñada para soportar los principales elementos del lenguaje VAGAX, entre los cuales se encuentran:

- Declaración y asignación de variables
- Expresiones aritméticas
- Estructuras de control de flujo
- Definición de funciones
- Llamadas a funciones
- Evaluación de expresiones

A continuación se presentan los componentes principales de la gramática.

8.1 Estructura general del programa

Un programa en VAGAX está compuesto por una secuencia de sentencias que se ejecutan de forma secuencial. Esta estructura se define mediante la siguiente regla gramatical:

```
7  program
8      : statement* EOF
9      ;
10
```

La regla indica que un programa puede contener cero o más sentencias, finalizando con el token de fin de archivo (EOF). Esto permite que el lenguaje procese múltiples instrucciones dentro de un mismo archivo.

8.2 Definición de sentencias

Las sentencias representan las unidades básicas de ejecución dentro del lenguaje. Estas incluyen declaraciones de variables, asignaciones, estructuras de control, llamadas a funciones y expresiones.

La gramática define las sentencias de la siguiente manera:

```
statement
    : varDecl SEMI
    | assignment SEMI
    | ifStatement
    | whileStatement
    | forStatement
    | functionDecl
    | returnStmt SEMI
    | expr SEMI
    ;
```

Cada una de estas reglas representa una operación que el intérprete puede ejecutar. Algunas sentencias requieren el uso del símbolo ; para indicar el final de la instrucción.

8.3 Declaración de variables

VAGAX permite declarar variables utilizando tipos de datos definidos en el lenguaje. La declaración puede incluir o no una asignación inicial.

La gramática define esta estructura de la siguiente forma:

```
varDecl
    : type ID ASSIGN expr
    | type ID
    ;
```

Donde:

- type representa el tipo de dato declarado.
- ID corresponde al identificador de la variable.
- expr corresponde a la expresión que define el valor inicial.

Los tipos de datos soportados por la gramática incluyen:

```
type
    : INT
    | FLOAT
    | BOOL_T
    | STRING_T
    | LIST
    | MATRIX
    | DICC
    | DATAFRAME
    ;
```

Esto permite al lenguaje manejar diferentes estructuras de datos utilizadas en procesos de análisis y aprendizaje automático.

8.4 Asignación de valores

Además de la declaración, el lenguaje permite modificar el valor de una variable previamente definida mediante una operación de asignación.

```
assignment
    : ID ASSIGN expr
    ;
```

En esta regla, el identificador recibe el resultado de la evaluación de una expresión.

8.5 Estructuras de control

Las estructuras de control permiten modificar el flujo de ejecución del programa mediante condiciones o ciclos.

8.5.1 Condicional IF

```
ifStatement
    : IF LPAREN expr RPAREN block
      (ELSE block)?
      END
    ;
```

Esta regla permite ejecutar un bloque de instrucciones si una condición se evalúa como verdadera, y opcionalmente ejecutar un bloque alternativo en caso contrario.

8.5.2 Ciclo WHILE

```
whileStatement
    : WHILE LPAREN expr RPAREN block END
    ;
```

El ciclo whilevag ejecuta repetidamente un bloque de instrucciones mientras la condición especificada sea verdadera.

8.5.3 Ciclo FOR

```
forStatement
    : FOR LPAREN assignment SEMI expr SEMI assignment RPAREN block END
    ;
```

El ciclo forvag permite controlar iteraciones mediante tres componentes principales:

- inicialización
- condición
- actualización

8.6 Bloques de instrucciones

Los bloques permiten agrupar múltiples sentencias dentro de estructuras de control o funciones.

```
block
    : LBRACE statement* RBRACE
    ;
```

Esto significa que un bloque está delimitado por llaves { } y puede contener múltiples sentencias.

8.7 Definición de funciones

VAGAX permite a los usuarios definir funciones personalizadas para encapsular lógica y reutilizar código. La gramática define la estructura de una función de la siguiente forma:

```
functionDecl
    : FUNCTION ID LPAREN paramList? RPAREN block END
    ;
```

Una función puede contener:

- un identificador que representa el nombre de la función
- una lista opcional de parámetros
- un bloque de instrucciones que define su comportamiento

La lista de parámetros se define mediante la siguiente regla:

```
paramList
    : ID (COMMA ID)*
    ;
```

Esto permite definir funciones con uno o múltiples parámetros.

8.8 Retorno de valores

Las funciones pueden devolver un resultado mediante la instrucción returnvag.

```
returnStmt
    : RETURN expr
    ;
```

Esta regla indica que la función retornará el valor obtenido al evaluar la expresión especificada.

8.9 Expresiones

Las expresiones constituyen uno de los componentes fundamentales del lenguaje, ya que permiten realizar operaciones matemáticas, comparaciones y evaluaciones lógicas.

La gramática define las expresiones de la siguiente manera:

```
expr
    : expr POW expr
    | expr MUL expr
    | expr DIV expr
    | expr MOD expr
    | expr PLUS expr
    | expr MINUS expr
    | expr EQ expr
    | expr NEQ expr
    | expr LT expr
    | expr GT expr
    | expr LE expr
    | expr GE expr
    | expr AND expr
    | expr OR expr
    | NOT expr
    | LPAREN expr RPAREN
    | functionCall
    | literal
    | ID
    ;
```

Esta estructura permite evaluar:

- operaciones aritméticas
- comparaciones
- operaciones lógicas
- llamadas a funciones
- valores literales

8.10 Llamadas a funciones

Las funciones pueden invocarse dentro de expresiones mediante la siguiente regla:

```
116     functionCall
117         : ID LPAREN argList? RPAREN
118         ;
119
```

Los argumentos enviados a la función se definen mediante:

```
120     argList
121         : expr (COMMA expr)*
122         ;
123
```

Esto permite enviar múltiples valores como parámetros en una llamada de función.

8.11 Literales

Los valores constantes utilizados dentro de las expresiones se reconocen mediante la siguiente regla:

```
126     literal
127         : NUMBER
128         | STRING
129         | BOOL_LIT
130         ;
```

Esto permite trabajar con valores numéricos, cadenas de texto y valores booleanos dentro del lenguaje.

En conjunto, esta gramática define la estructura sintáctica del lenguaje VAGAX y permite que el parser generado por ANTLR v4 construya un árbol sintáctico que posteriormente será interpretado por el sistema de ejecución implementado en Python. De esta manera, se establece una base formal para la interpretación de programas escritos en el lenguaje, garantizando consistencia y extensibilidad dentro del diseño del DSL.

9. FUNCIONES

Esta sección describe la implementación del sistema de funciones del lenguaje VAGAX, el cual permite encapsular lógica, reutilizar código y organizar programas de manera modular.
La implementación de esta funcionalidad se realizó utilizando ANTLR v4 para el análisis sintáctico del lenguaje y Python para la ejecución del intérprete, permitiendo declarar funciones definidas por el usuario, recibir parámetros y retornar valores.

La implementación del sistema de funciones se compone de tres elementos principales:

- Definición de la gramática de funciones en gramatica.g4
- Ejecución de funciones en el intérprete (interpreter.py)
- Integración con el flujo de ejecución del lenguaje mediante main.py

9.1 . Definición de funciones en la gramática (gramatica.g4)

La gramática del lenguaje define la estructura que deben seguir las funciones dentro de VAGAX.
Las funciones se declaran utilizando la palabra reservada FNVAG, seguida del nombre de la función, la lista de parámetros y un bloque de instrucciones que finaliza con una sentencia RETURNVAG.

```
functionDecl
    : 'FNVAG' ID '(' paramList? ')' '{' statement* returnStmt '}'
    ;
```

Esta regla permite que una función tenga:

- Un identificador (nombre de la función)
- Una lista opcional de parámetros
- Un bloque de instrucciones
- Una expresión de retorno

Ejemplo de función en vagax:

```
FNVAG suma(a,b){
RETURNVAG a + b;
}
```

9.2.1. Manejo de parámetros de funciones

Las funciones pueden recibir múltiples parámetros.

La gramática define la lista de parámetros mediante la siguiente regla:

```
paramList
    : ID (',' ID)*
    ;
```

Parte	Qué es	Qué hace
ID	Token léxico	Reconoce cualquier nombre de variable (a, b, x, peso, etc.)
','	Terminal literal	Reconoce exactamente el carácter coma
(',' ID)	Grupo	Reconoce el par "coma seguida

		de un ID"
*	Cuantificador	Significa **cero o más veces**

Eso permite definir funciones como

```
FNVAG multiplicar( a , b , c )
                   ↑   ↑   ↑
                   |   |   └── segundo (',' ID): coma + c  ← el * lo repite
                   |   └────── primer  (',' ID): coma + b  ← el * lo repite
                   └──────────── ID obligatorio: a
```

Cada parámetro será posteriormente asociado con los valores enviados durante la llamada de la función.

- Llamada de funciones

El lenguaje permite invocar funciones previamente definidas mediante una expresión de llamada.

```
functionCall
    : ID '(' argList? ')'
    ;
```

Ejemplo de llamada:

```
valor = suma(4,5);
```

- Implementación del intérprete de funciones (interpreter.py)

La ejecución de las funciones se implementa en el archivo interpreter.py, el cual contiene la lógica necesaria para almacenar funciones, evaluar sus parámetros y ejecutar su cuerpo.

El intérprete mantiene dos estructuras principales:

self.variables = {}
self.functions = {}

- **variables:** almacena las variables del programa

- **functions:** almacena las funciones definidas por el usuario

```python
# ──────── PROGRAMA ────────

def visitProgram(self, ctx):
    for stmt in ctx.statement():
        self.visit(stmt)

# ──────── STATEMENT ────────

def visitStatement(self, ctx):

    if ctx.expr():
        result = self.visit(ctx.expr())

        if result is not None:
            print(result)

        return result

    return self.visitChildren(ctx)

# ──────── BLOQUE ────────

def visitBlock(self, ctx):

    for stmt in ctx.statement():
        self.visit(stmt)

# ──────── DECLARACIÓN DE VARIABLES ────────

def visitVarDecl(self, ctx):

    name = ctx.ID().getText()

    if ctx.expr():
        value = self.visit(ctx.expr())
    else:
        value = None

    self.variables[name] = value
```

visitProgram

Es el primer método que se ejecuta cuando corre un programa VAGAX. Simplemente recorre todas las sentencias del programa una por una y llama self.visit(stmt) sobre cada una, lo que dispara el método visitor correspondiente según el tipo de sentencia

visitStatement

Es el método que decide qué hacer con cada sentencia. Se hace una sola pregunta: ¿esta sentencia es una expresión? Si lo es, la evalúa con self.visit(ctx.expr()) y si el resultado no es None lo imprime. Si no es una expresión, delega a visitChildren para que ANTLR siga bajando por el árbol y encuentre el visitor correcto.

visitBlock

Hace exactamente lo mismo que visitProgram pero para un bloque delimitado por llaves { }. Recorre todas las sentencias internas y visita cada una. Se usa dentro de funciones, condicionales y ciclos.

visitVarDecl

Es el método que maneja líneas como x = 10;. Funciona en dos pasos: primero extrae el nombre de la variable con ctx.ID().getText(), luego revisa si hay una expresión del lado derecho. Si la hay, la evalúa y guarda el resultado; si no hay nada, guarda None. Al final registra el par nombre-valor en self.variables, que es el diccionario global de variables del intérprete.

```
# operaciones
if ctx.getChildCount() == 3:

    left = self.visit(ctx.expr(0))
    right = self.visit(ctx.expr(1))
    op = ctx.getChild(1).getText()

    if op == '+':
        return left + right

    if op == '-':
        return left - right

    if op == '*':
        return left * right

    if op == '/':
        return left / right

    if op == '%':
        return left % right

    if op == '^':
        return left ** right

    if op == '==':
        return left == right

    if op == '≠':
        return left ≠ right

    if op == '<':
        return left < right

    if op == '>':
        return left > right

    if op == '≤':
        return left ≤ right

    if op == '≥':
        return left ≥ right

    if op == '&&':
        return left and right

    if op == '||':
```

callFunction — ejecutar una función
Es el método que se activa cuando el intérprete encuentra algo como multiplicar(2, 3, 4). Sigue estos pasos en orden:

Busca la función por nombre en self.functions. Si no existe, lanza un error inmediatamente.
Evalúa los argumentos recorriendo la lista argList del árbol. Para multiplicar(2, 3, 4) esto produce la lista [2, 3, 4].

Crea las variables locales combinando parámetros con argumentos mediante zip. Si params = ['a','b','c'] y args = [2,3,4], entonces local_vars = {'a':2, 'b':3, 'c':4}.

Sustituye temporalmente el entorno de variables guardando las variables actuales en old_vars y reemplazándolas por una mezcla de las variables globales más las locales. Esto es lo que permite que dentro de la función a, b y c tengan los valores correctos.

Ejecuta el cuerpo línea por línea con walk, y al final evalúa el RETURNVAG para obtener el resultado.

Restaura las variables originales antes de retornar, para que los cambios dentro de la función no afecten el entorno exterior.

def walk
Es un método auxiliar más simple. Su único trabajo es recorrer todos los hijos de un nodo del árbol sintáctico y llamar accept sobre cada uno que lo tenga. Esto es lo que activa el patrón Visitor: accept le dice a ANTLR "visita este nodo con el intérprete", disparando el método visit correspondiente según el tipo de nodo.

```
callFunction("multiplicar", 2, 3, 4)
    ↓
crea entorno local: {a:2, b:3, c:4}
    ↓
walk(cuerpo)  →  visita cada sentencia
    ↓
evaluate(RETURNVAG a*b*c)  →  2*3*4  →  24
    ↓
restaura entorno original
    ↓
retorna 24
```

10. Ejecución de llamadas a funciones

- Cuando el intérprete encuentra una llamada a una función, realiza el siguiente proceso:
- Buscar la función en la tabla de funciones.
- Evaluar los argumentos enviados en la llamada.
- Asociar los argumentos con los parámetros definidos en la función.
- Ejecutar el bloque de instrucciones de la función.
- Evaluar la expresión definida en RETURNVAG.
- Devolver el resultado al contexto donde fue llamada la función.

```
FNVAG suma(a,b){
RETURNVAG a + b;
}

valor = suma(4,5);
PRINTVAG(valor);
```

salida

```
┌──(venv)─(kali㉿kali)-[~/vagax]
└─$ python main.py
9
```

10.1 Integración con el sistema de ejecución (main.py)

- El archivo main.py actúa como el punto de entrada del programa y se encarga de conectar el parser generado por ANTLR con el intérprete.
- El flujo de ejecución es el siguiente:
- Se lee el archivo que contiene el programa escrito en VAGAX.
- El lexer convierte el texto en tokens.
- El parser genera el árbol sintáctico del programa.
- El intérprete recorre el árbol utilizando ParseTreeWalker.
- Durante ese recorrido se detectan las declaraciones y llamadas a funciones, ejecutando la lógica implementada en el intérprete.
- Este proceso permite ejecutar programas escritos en el lenguaje VAGAX que utilicen funciones definidas por el usuario.

11. Conclusiones:

- El desarrollo de VAGAX permitió establecer una propuesta formal para un Lenguaje de Dominio Específico orientado a procesos de Machine Learning y Deep Learning, definiendo sus principios de diseño, su estructura general y los componentes principales que lo conforman. Esto demuestra que el proyecto cuenta con una base conceptual sólida para continuar con su implementación en etapas posteriores.

- A través del diseño del lenguaje se logró identificar la importancia de contar con una sintaxis especializada, clara y cercana al dominio, capaz de representar operaciones matemáticas, matriciales, de control de flujo y de aprendizaje automático de manera más comprensible que en un lenguaje de propósito general. En este sentido, VAGAX se plantea como una herramienta que busca mejorar la legibilidad, modularidad y organización de este tipo de procesos.

- El planteamiento arquitectónico de VAGAX, apoyado en ANTLR v4, Python y un runtime interno modular, permite proyectar una implementación escalable y coherente con los objetivos del proyecto. Aunque en esta primera etapa el énfasis estuvo en el diseño del lenguaje, se establecieron las bases necesarias para avanzar hacia futuras fases de desarrollo, validación e integración de funcionalidades más complejas dentro del dominio del aprendizaje profundo.

12. Referencias

Pineda Cardona, B., & Rivera Maldonado, J. H. (2015). *Análisis, diseño e implementación de un prototipo de un lenguaje de dominio específico (DSL) UTP*

Pérez Borrero, I., & Gegúndez Arias, M. E. (2021). Deep learning: Fundamentos, teoría y aplicación. Universidad de Huelva.

Deulofeu Gómez, R. (2020). Introducción y optimización estocástica de redes neuronales profundas MLP , Universitad de Barcelona